

NAME : Syed Salman .S

Reg No:192524076

Subject Name :Data Structures

Subject code:0316

ASSESSMENT TOLL-C2(AT-1)

37. Assume, a single character is present in the data field of DLL with 6 nodes as

S<=>I<=>M<=>A<=>T<=>S. Write a function to print the same in reverse order.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node
```

```
{
```

```
    char data;
```

```
    struct Node *prev;
```

```
    struct Node *next;
```

```
};
```

```
// Function to create a new node
```

```
struct Node* createNode(char ch)
```

```
{
```

```
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = ch;
```

```
    newNode->prev = NULL;
```

```
    newNode->next = NULL;
```

```

        return newNode;
    }

// Function to print DLL in reverse order
void printReverse(struct Node *head)
{
    struct Node *temp = head;

    // Move to the last node
    while(temp->next != NULL)
        temp = temp->next;

    printf("Reverse Order: ");

    // Traverse backwards
    while(temp != NULL)
    {
        printf("%c ", temp->data);
        temp = temp->prev;
    }
}

int main()
{
    struct Node *head, *n2, *n3, *n4, *n5, *n6;

    head = createNode('S');
    n2 = createNode('I');
    n3 = createNode('M');
    n4 = createNode('A');
    n5 = createNode('T');
    n6 = createNode('S');

```

```
head->next = n2;
n2->prev = head;
n2->next = n3;
n3->prev = n2;
n3->next = n4;
n4->prev = n3;
n4->next = n5;
n5->prev = n4;
n5->next = n6;
n6->prev = n5;

printReverse(head);

return 0;
}
```

Assume a single character is present in the data field of a Single Linked List with 6 nodes as S- > I -> M -> A -> T -> S. Write a function to print the same in reverse order using stack data structure.

Run

Debug

Stop

Share

Save

Beautify

Language C

main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     char data;
7     struct Node *prev;
8     struct Node *next;
9 };
10
11 // Function to create a new node
12 struct Node* createNode(char ch)
13 {
14     struct Node *newNode = (struct Node*) malloc(sizeof(struct Node));
15     newNode->data = ch;
16     newNode->prev = NULL;
17     newNode->next = NULL;
18     return newNode;
19 }
20
21 // Function to print DLL in reverse order
22 void printReverse(struct Node *head)
23 {
24     struct Node *temp = head;
25
26     // Move to the last node
27     while(temp->next != NULL)
28         temp = temp->next;
29
30     // Print the data of the last node
31     printf("%c", temp->data);
32
33     // Move to the previous node
34     while(temp->prev != NULL)
35     {
36         temp = temp->prev;
37         printf("%c", temp->data);
38     }
39 }
```

input

Reverse Order: S T A M I S

...Program finished with exit code 0
Press ENTER to exit console.

38. Assume a single character is present in the data field of a Single Linked List with 6 nodes as S-> I-> M-> A-> T-> S. Write a function to print the same in reverse order using stack data structure.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 10
```

```
// Structure for Singly Linked List
```

```
struct Node
```

```
{
```

```
    char data;
```

```
    struct Node *next;
```

```
};
```

```
// Stack
```

```
char stack[MAX];
```

```
int top = -1;
```

```
// Push operation
```

```
void push(char ch)
```

```
{
```

```
    if (top == MAX - 1)
```

```
    {
```

```
        printf("Stack Overflow\n");
```

```
        return;
```

```
    }
```

```
    stack[++top] = ch;
```

```
}
```

```
// Pop operation
```

```
char pop()
```

```
{  
    if (top == -1)  
    {  
        return '\0';  
    }  
    return stack[top--];  
}
```

```
// Function to print linked list in reverse using stack
```

```
void printReverse(struct Node *head)
```

```
{  
    struct Node *temp = head;
```

```
    // Push all node data into stack
```

```
    while (temp != NULL)
```

```
{  
    push(temp->data);  
    temp = temp->next;  
}
```

```
    // Pop and print
```

```
    printf("Reverse Order: ");
```

```
    while (top != -1)
```

```
{  
    printf("%c ", pop());  
}  
}
```

```
// Main Function
```

```
int main()
```

```

{
    struct Node *head, *n2, *n3, *n4, *n5, *n6;

    // Create nodes
    head = (struct Node *)malloc(sizeof(struct Node));
    n2 = (struct Node *)malloc(sizeof(struct Node));
    n3 = (struct Node *)malloc(sizeof(struct Node));
    n4 = (struct Node *)malloc(sizeof(struct Node));
    n5 = (struct Node *)malloc(sizeof(struct Node));
    n6 = (struct Node *)malloc(sizeof(struct Node));

    // Store data
    head->data = 'S';
    n2->data = 'I';
    n3->data = 'M';
    n4->data = 'A';
    n5->data = 'T';
    n6->data = 'S';

    // Link nodes
    head->next = n2;
    n2->next = n3;
    n3->next = n4;
    n4->next = n5;
    n5->next = n6;
    n6->next = NULL;

    // Print in reverse
    printReverse(head);

    return 0;
}

```

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define MAX 10
5
6 // Structure for Singly Linked List
7 struct Node
8 {
9     char data;
10    struct Node *next;
11 };
12
13 // Stack
14 char stack[MAX];
15 int top = -1;
16
17 // Push operation
18 void push(char ch)
19 {
20     if (top == MAX - 1)
21     {
22         printf("Stack Overflow\n");
23         return;
24     }
25     stack[++top] = ch;
26 }
27
28 // Pop operation
```

Reverse Order: S T A M I S

...Program finished with exit code 0
Press ENTER to exit console.